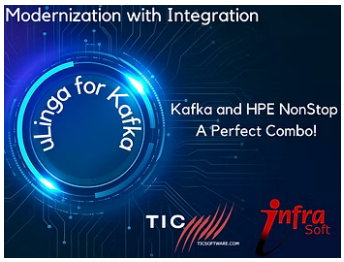
# Kafka and HPE NonStop – A Perfect Combo!

Updated: Mar 26, 2024




## INTRODUCTION

One of the best ways to modernize NonStop is to integrate its applications with other software, platforms and cloud services. This interoperability is extremely important as it will extend the value of the NonStop applications, and underscore the flexibility of the platform in the enterprise. Over the past few years, one of the most popular tools that have been adopted by many corporations is Apache Kafka. To date, it has been reported that over 20,000 companies in the US alone are using Kafka for real time stream processing, messaging, queueing and other applications. Integration with Kafka will open NonStop to interoperate with many different platforms and applications.

The purpose of this article is to provide an overview of Kafka's architecture and benefits, and to show how uLinga for Kafka from Infrasoft can enable NonStop applications to interoperate with the Kafka ecosystem.

## History

Kafka was originally developed at LinkedIn by Jay Kreps, Neha Narkhede and Jun Rao Jay. The original project aimed to provide a unified, high-throughput, low-latency platform for handling real-time data feeds. Kreps chose to name the software after the author Franz Kafka because it is "a system optimized for writing", and he liked Kafka's work. It was subsequently open sourced in early 2011 and became a fully fledged Apache project in October 2012. While Apache Kafka is an open-source project, enterprise support is backed up by Confluent, a company founded by the product's original developers. The Confluent Platform is a complete Kafka distribution, with additional community and commercial enhanced features.

## Kafka architecture

Kafka is a distributed publish-subscribe messaging system and a robust queue that can handle a high volume of data. Kafka messages are persisted on the disk and can be replicated to provide fault tolerance. It works very well for real-time streaming applications, as well as data distribution to other platforms.

Kafka supports low latency message delivery and gives guarantee for fault tolerance in the presence of machine failures. Kafka is very fast, with the ability to support millions of messages/sec. Kafka persists all data to the disk, utilizing a very efficient transfer method to achieve a high throughput.

Kafka Terminology

| | |
|---|---|
| Topic | A category of message in Kafka. Analogous to a file or SQL table. |
| Partition | A Topic can be stored in multiple partition units. Analogous to a file or table partition. |
| Broker | A server that houses one or more partitions. |
| Producer | Clients or applications that publish messages to one or more Kafka Topics. |
| Consumer | Applications that subscribe to one or more Topics and read the messages. |
| Kafka Cluster | A Kafka cluster consists of one or more servers (Kafka brokers) running Kafka. |

Categories

All Posts (97)
Amazon Web Services (1)
API Gateway (8)
Cloud (7)
Business Intelligence (19)
Application Development (9)
Development (7)
Gateway (5)
iTP Web Server (5)
LogWatch (13)
Java (11)
LightWave (8)
Methodology (3)
OSS (10)
Messaging (8)
Modernization (57)
Operation (6)
REST (15)
Security (2)
Cypress (1)
SCOBOL (5)
SOAP (8)
COBOL (21)
SOG (2)
XML (10)
OOP (1)
TAG (3)
CLIM (2)
netsh (1)
uLinga (4)
TeleFTP (3)
JSON (2)
Tandem (8)
TOP (7)
TelePath (2)

TeleFax (2)
MDA (5)
TeleMail (4)
Report.Web (1)
Tools (17)
Thunder Suite (5)
Web Services (18)
Training (14)
uLinga (1)
TIC Navigator (6)
TAL (4)
TACL (4)
C/C++ (4)
GenAI (6)
LLM (1)
Nonstop (82)

Kafka has Producers which publish messages to a Topic, and Consumers which pull the Topic data as subscribers. You can partition a Topic across multiple Brokers to allow parallel production or consumption. This enables the load to be balanced across multiple partitions and brokers for higher performance. Similarly, a single broker can house one or more such partitions, allowing flexibility to balance workload. All messages written to Kafka are persisted on disk and can be replicated to multiple Brokers for fault tolerance. In addition, you can configure messages to be stored for a configurable period of time, for days or weeks or longer.

## Why use Kafka?

Here is an example of the challenges that an enterprise might encounter when trying to integrate data from different sources, including application logs, events, relational databases, OLTP transactions, CSV files, search systems, monitoring systems, OLAP stores, etc. Each of these needs data feeds in a distributed environment. The result is a total mess that is difficult to implement and impossible to manage.

## Kafka solution: Publish once, consumed by many

Kafka solves this problem by acting as a kind of universal data connector. Each system can feed into this central pipeline (producer) or be fed by it (consumer). Applications or stream processors can tap into it to create new data streams, when then can be fed back into the various systems for serving. Continuous feeds of well-formed data act as a kind of standard across different systems, applications, and data repositories. The result is a high-performance architecture that is simple, scalable and more manageable

## Kafka Benefit – High Performance

Kafka has high throughput for both publishing and subscribing messages, being able to support up to millions of messages/sec. Kafka takes advantage of multiple partitions to achieve parallel processing. Also, a Kafka consumer group is a group of consumers that can process the same Topic in parallel. This enables the record processing to be load-balanced over the consumer instances. This is similar to the high-performance architecture achieved on NonStop via the use of partitioned disks and application server classes scaled across multiple CPUs.

Another contributing factor to Kafka's high performance is its Zero Copy feature. Kafka makes use of this zero-copy principal by requesting the operating system to move the data directly to the network rather than moving it via the application. This greatly reduces the number of context switches and speeds up the operation drastically.

All data published to Kafka is written to disk. Kafka provides fail-over capability or fault tolerance for a topic via data replication. Replication is accomplished by copying the partition data to other brokers, also known as replicas. Imagine this is like NonStop disk processes that provide automatic fault tolerance via replicating data between mirrored Primary and Backup disks. But Kafka takes it to the next level by allowing more than one copy of backup data. You can specify how many copies of the partition you need via a configuration parameter called Replication Factor. So, a partition having a replication factor of 5 will guarantee there be five copies of the data residing in different brokers for redundancy. If a "primary" Broker (Leader) where the Topic resides goes down, Kafka automatically selects one of the replicas to stand in and continue processing without losing consistency.

## Kafka Benefit – Guarantees Message Order

Kafka preserves the order of messages within a partition. This means that if messages were sent from the Producer in a specific order, they will be guaranteed to be written to a partition in that exact same order. The result is that all the consumers will be able to read and process them in that same order. This can be a critical feature for some applications which demand the messages or transactions must be processed in order. One example is database replication, which requires that change data capture (CDC) transactions be processed in order. Similarly, in a payment environment, transaction log files often contain the delta of a transaction (withdraw $50 or deposit $100 for example) so the order that these transactions are fed into the core banking application can be critical.

## uLinga for Kafka Features

uLinga for Kafka takes a distinctly NonStop approach to Kafka integration: it runs as a natively compiled Guardian (or OSS) process pair, and supports the Kafka communications protocols directly over TCP/IP. This removes the need for Java libraries or intermediate servers and databases, providing the best possible performance on NonStop. It also allows uLinga for Kafka to directly communicate with the Kafka cluster, ensuring data is streamed as quickly and reliably as possible.

Other NonStop Kafka integration solutions require an interim application and/or database, generally running on another platform. This can be less than ideal as that additional platform may not have the reliability of the NonStop, and could introduce a single point of failure. It can also increase latency, in terms of getting the data into Kafka as quickly as possible.

## Features

uLinga for Kafka supports the same access points available with other uLinga products to enable applications to stream their data to Kafka. These include Inter-Process Communication (IPC), Pathsend and HTTP/REST. IPC and Pathsend interfaces allow NonStop applications to open uLinga for Kafka and send data via Guardian IPC, or via a Pathsend message. The HTTP/REST interface allows applications on the NonStop or on any other platform, such as API gateways, to stream via HTTP to Kafka. uLinga for Kafka also introduces a new access point which allows for Enscribe files to be monitored and read in real-time as they are being written by an application.

### Entry-Sequenced and Relative Enscribe File Support

Entry-sequenced or relative files (such as a transaction log file from a payment application, or an access log for an in-house application), can be supported by uLinga for Kafka. uLinga for Kafka performs an "end of- file chase" whereby new records are read from the Enscribe file as they are written by the application, and immediately streamed to the Kafka cluster. uLinga for Kafka monitors for the creation of new files and automatically picks them up and begins processing them.

### Pathsend and Guardian IPC Support

Applications can also explicitly send data via the Pathsend and IPC interfaces provided by uLinga for Kafka. This might be useful where specific data streams need to be generated and sent directly from the application. A Pathsend client, such as a Pathway Requestor or Pathway Server, simply sends the relevant data via a Pathsend request to uLinga. A NonStop process sends the data to uLinga for Kafka via IPC calls. uLinga for Kafka's support for the native Kafka protocols ensures that this data will be streamed with the lowest latency possible. This same interface can also be used to send any Enscribe files and other NonStop to Kafka, via utilities such as FUP and EMSDIST. An EMSDIST can be run, for instance, to easily stream EMS events to Kafka via the uLinga for Kafka IPC interface.

## NonStop Application Streaming via HTTP/REST

uLinga for Kafka's HTTP interface can be used by any HTTP Client, on NonStop, or any other platform. This might be useful with API gateways, such as the new product recently launched by HPE. API gateways, with the central position they occupy within the enterprise, will often produce data that will need to be streamed to Kafka. Administrators of an API gateway, which of course is inherently REST-capable, might decide to utilize the REST interface into uLinga for Kafka – it's simple, it's well-understood, and it will work "out of the box" with a REST-capable client.

### uLinga for Kafka provides the additional following key features:

- Online configuration via the built- in browser UI WebCon, allowing for simple management of uLinga processes and resources.
- Extensive tracing via uLinga's tracing capabilities and standard logging support
- Interactive command line interface and REST-based command interface to allow command and control from any environment
- Kafka Exactly Once Semantics support, along with traditional NonStop fault-tolerance, provide unparalled reliability and ensure that every transaction is streamed to Kafka
- Superior performance – uLinga's well-proven C code-base and NonStop optimisations allow for

transaction rates of 25,000 TPS and higher.

## How uLinga for Kafka Can Benefit NonStop Applications

### Kafka Connectors

One major benefit of using Kafka is the extensive number of off-the-shelf connectors available for different third-party products. Kafka Connectors are ready-to-use components, which can help you import data from external systems into Kafka topics (source connectors) and export data from Kafka topics into external systems (sink connectors). Some are commercial products, while others are open source. Regardless of your application needs, chances are there is already an available connector for the application you want to integrate with. You just need to publish your message into Kafka, and the connector will do the rest.

You can take advantage of these available connectors by using uLinga to publish your NonStop data or messages to Kafka, which can be consumed by any number of sink connectors of your choice.

### Use Case: Log Analysis

With uLinga, you can send NonStop application logs or system events to Kafka for real time analysis by off-platform SIEM (Security Information and Event Management) packages, log analysis tools like Splunk, or observability platform like New Relic One.

### Use Case: Database replication

uLinga can work in concert with database replication products like Gravic's Shadowbase to send NonStop changed data to Kafka for use by other database platforms like SQL Server or Oracle.

### Use Case: Application Integration

uLinga enables your NonStop applications to easily generate workflow events or send messages to applications running on other platforms like Windows or Linux, or even third-party packages like SAP. An example could be a Base24 transaction on NonStop resulting in a message being sent to another system for back-office reconciliation. Kafka is increasingly being used as a foundation for fraud detection systems, and with real-time transaction data being sent by uLinga, that data can easily be integrated with Kafka-based fraud detection systems. All these messages can be delivered by Kafka in a reliable and asynchronous manner, and not slow down processing on the NonStop independent of the target system consumers. Your NonStop application can accomplish that by sending an IPC message to uLinga, or simply writing a record to a file being monitored by uLinga. This allows NonStop to seamlessly integrate well with the rest of the enterprise environment.

## Conclusion

Hopefully this article has given you an idea of how Kafka works, what is possible with it, and how you might integrate and modernize your NonStop applications and data by integrating with Kafka. If you'd like to learn more about uLinga for Kafka, please get in touch – it's easy to arrange a demo or Proof of Concept, to allow you to see how it performs in your environment.

Read more Blogs!

NonStop Integration – LogWatch and uLinga

How To Aggregate Nonstop Logs To Kafka In One Simple Step

Contact us:

phil_ly@ticsoftware.com | Tel. 516-466-7990 | follow us on twitter.com/TICSOFTWARE

---

Feedback please

Do you find this tutorial blog helpful? Let us know what you think, and how we can make it even better. Don't forget, you can subscribe to our blogs to get automatic email notification when a new blog is available.

---

Phil Ly is the president of TIC Software, a leading provider of software and services for the NonStop community focused on NonStop application modernization. Under Phil's guidance, TIC Software has a proven track record of helping clients seamlessly integrate NonStop with next-generation technologies. A recognized technical leader, Phil is passionate about empowering organizations to unlock the full potential and longevity of their NonStop platform through innovative solutions.

About Us

Who We Are

TIC Team

Partners

Clients

Products

Messaging

Gateway

Business Intelligence

Modernize

Services

Training

Consulting

Cloud

Resources

Success Stories

TIC News

TIC Blog

Contact Us

TIC Software
185 Great Neck Road
4th Floor, Suite 453
Great Neck, NY 11021

sales-support@ticsoftware.com

Technical Support
Phone: 516-466-7990
support@ticsoftware.com

© 2026 TIC Software Privacy Statement | Terms of Use